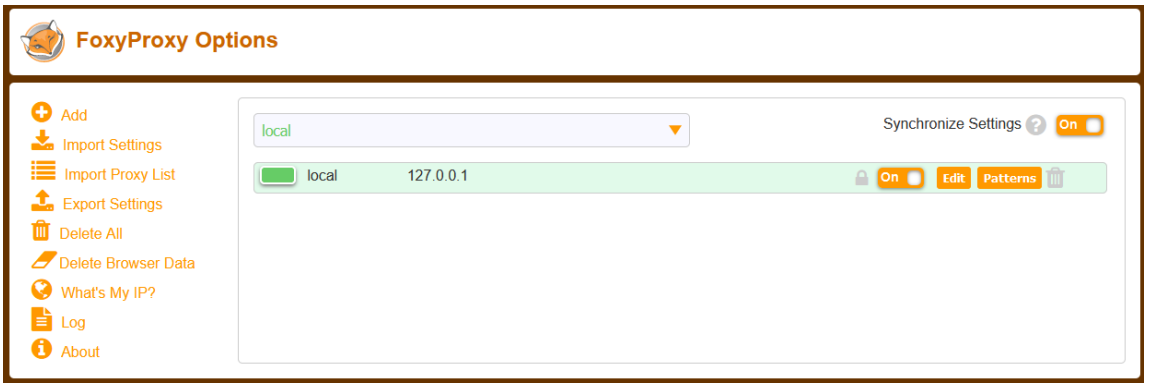
1 A1 Access Broken Control

Curso: Seguridad de Aplicaciones Web y Apis basado en Vulnerabilidades
OWASP Top 10:2025
Instructor: Carlos Carreño

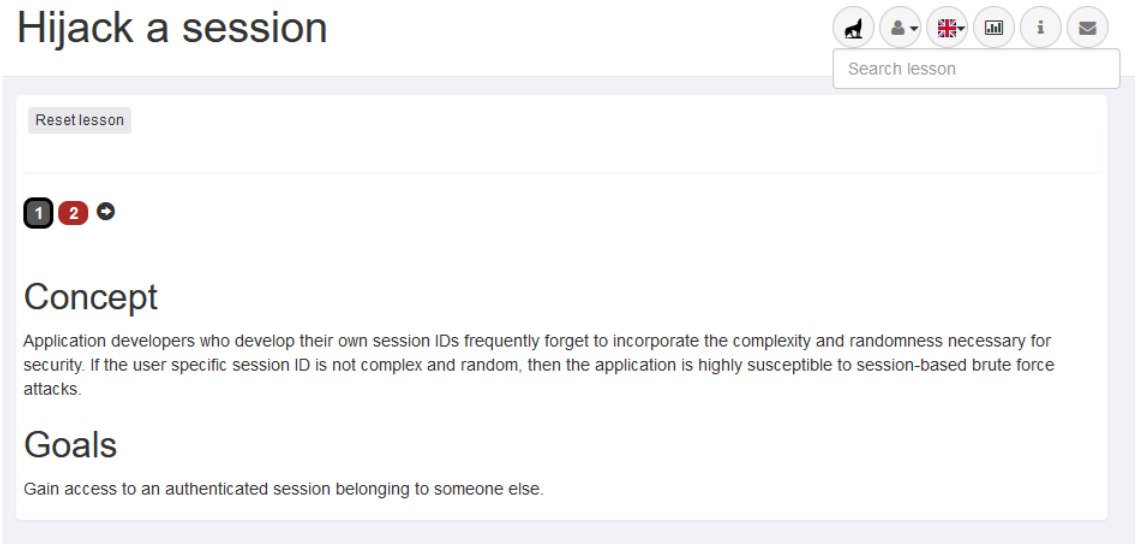
1.1 Hijack sesión

Los desarrolladores de aplicaciones que desarrollan sus propios ID de sesión con frecuencia olvidan incorporar la complejidad y aleatoriedad necesarias para la seguridad. Si el ID de sesión específico del usuario no es complejo ni aleatorio, entonces la aplicación es muy susceptible a ataques de fuerza bruta basados en sesiones

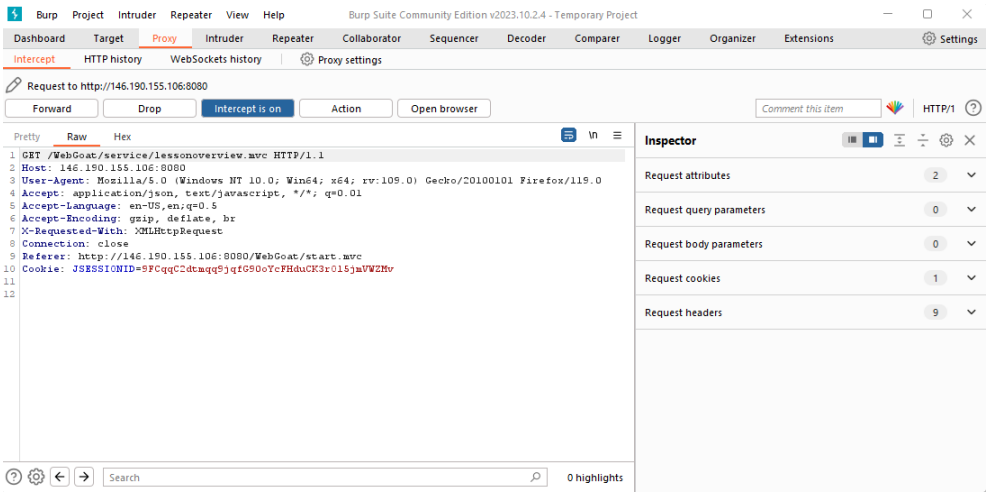
Asegúrate que FoxyProxy este activo en el navegador FireFox.



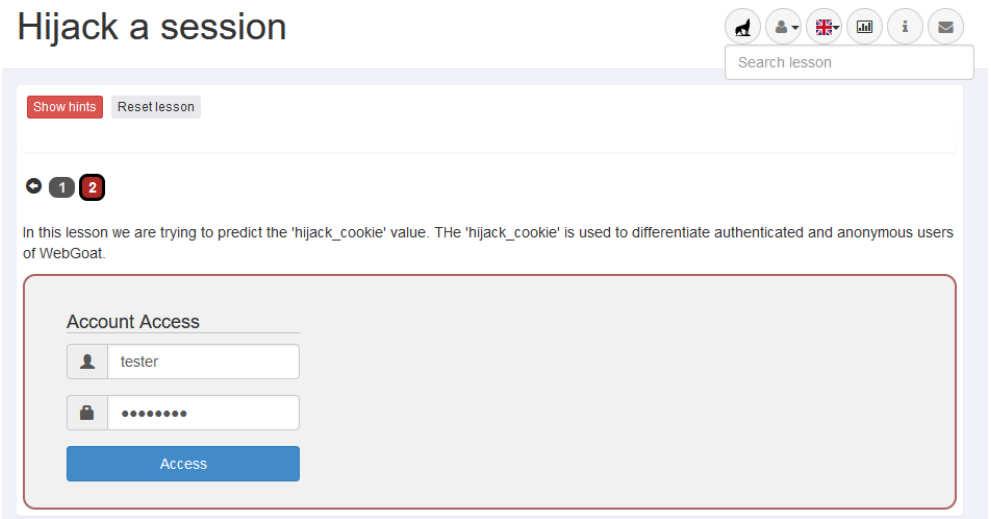
Captura el tráfico con el proxy Burp Suite.



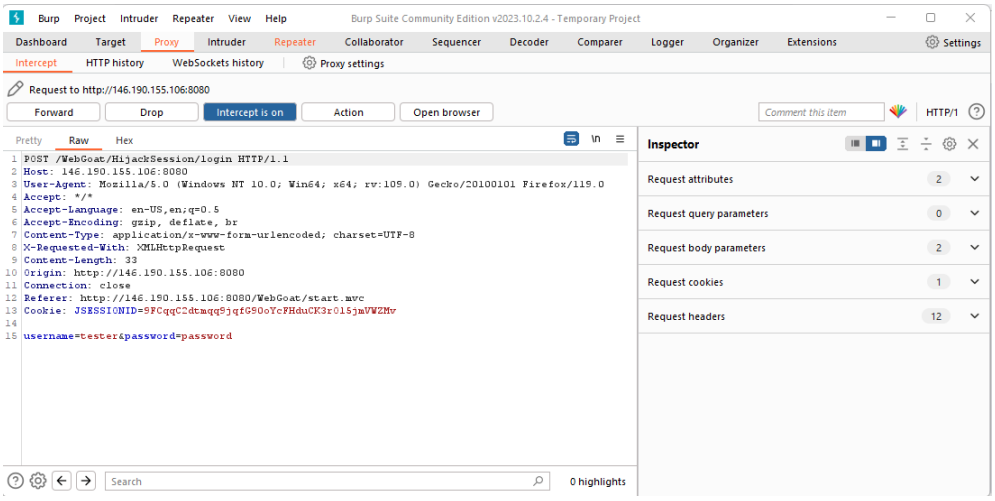
Abre Burp Suite.



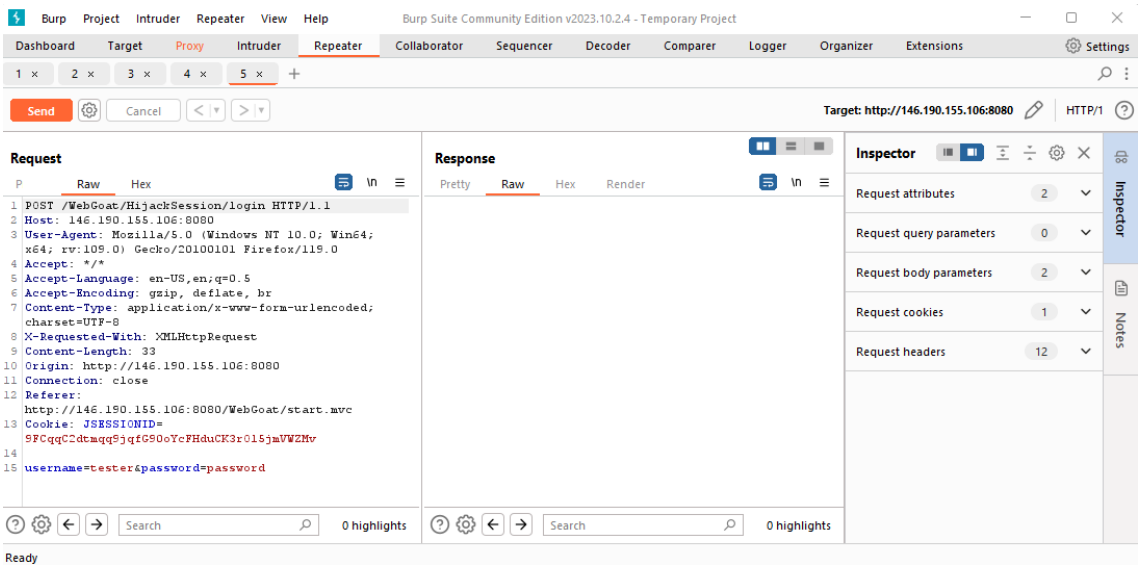
Intenta el login con el usuario actual (tester en este ejemplo).



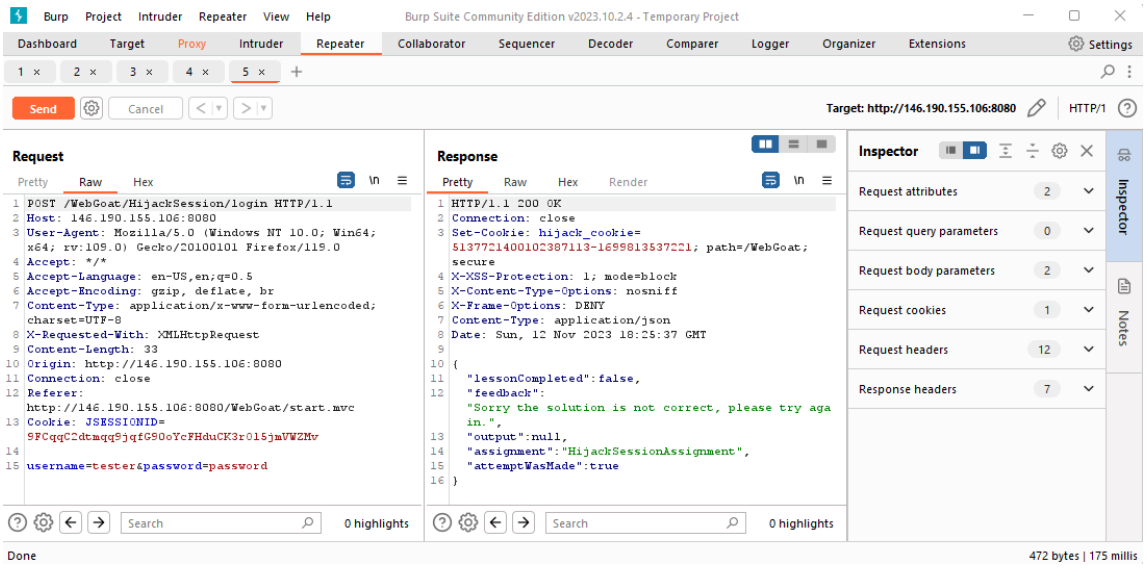
Captura en Blurp Suite la petición: “POST /WebGoat/HijackSession/login HTTP/1.1”



Envía a “**Repeater**” a petición capturada.



Haz Clic en “**Send**” para obtener el Cookie.



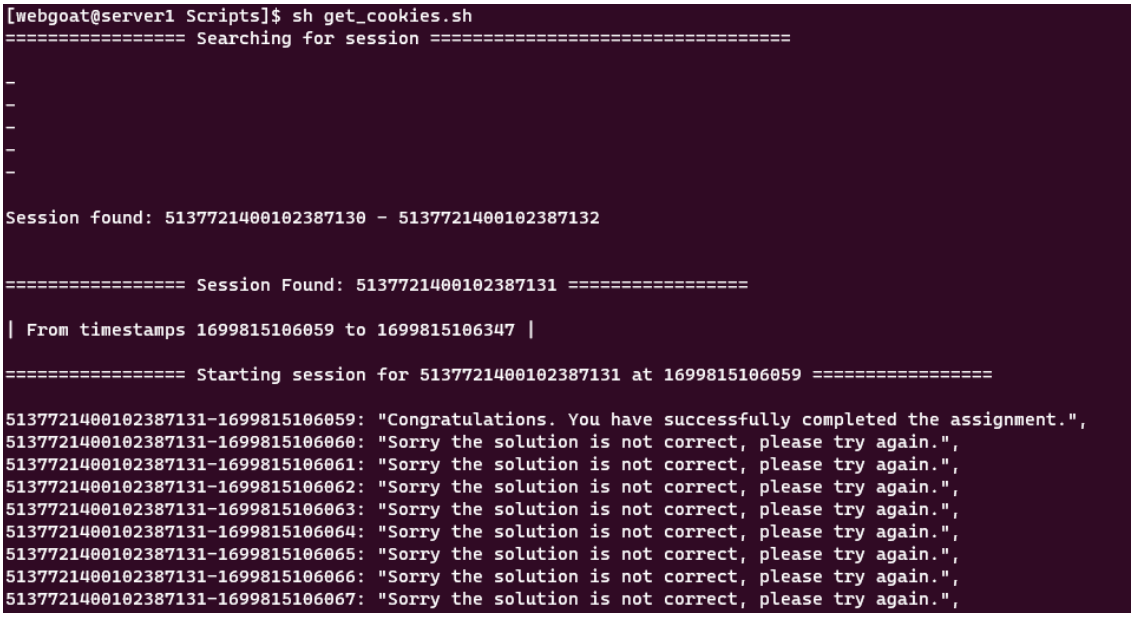
Edita el script get_cookies.sh. agrega el JSESSIONID que obtuviste con BlurpSuite.

```
# !/bin/bash

username=tester
password=password
JSESSIONID=9FCqqC2dtmqq9jqfG90oYcFHduCK3rOl5jmVWZMv

sessionFoundId=0
sessionFoundStartTime=0
sessionFoundEndTime=0
currentSessionId=0
previousSessionId=0
currentSessionTimestamp=0
previousSessionTimestamp=0
```

Ejecuta el script y verifica que retorne el mensaje de éxito.



1.2 Insecure Direct Object References

Las referencias directas a objetos se producen cuando una aplicación utiliza entradas proporcionadas por el cliente para acceder a datos y objetos.

Ejemplos

Los ejemplos de referencias directas a objetos que utilizan el método GET pueden parecerse a

`https://some.company.tld/dor?id=12345`

`https://some.company.tld/images?img=12345`

`https://some.company.tld/dor/12345`

Estos se consideran inseguros cuando la referencia no se maneja adecuadamente y permite eludir autorizaciones o revelar datos privados que podrían usarse para realizar operaciones o acceder a datos que el usuario no debería poder realizar o acceder. Digamos que, como usuario, vas a ver tu perfil y la URL se parece a:

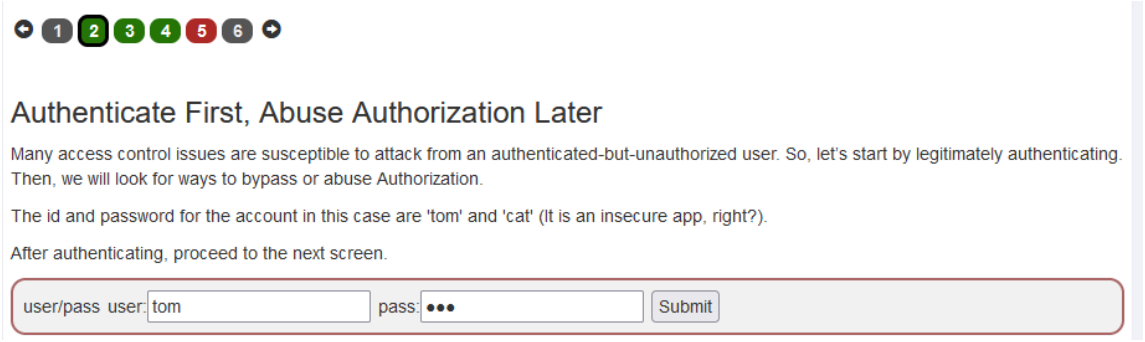
`https://some.company.tld/app/user/23398`

... y podrás ver tu perfil allí. ¿Qué sucede si navegas a:

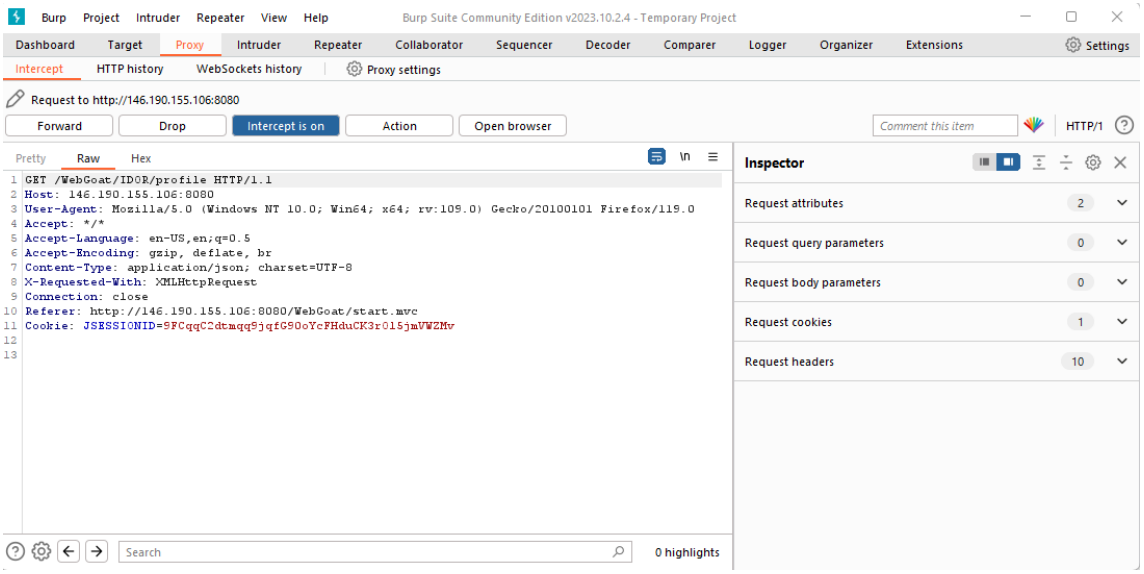
`https://some.company.tld/app/user/23399`…o use otro número al final. Si puede manipular el número (id de usuario) y ver el perfil de otra persona, entonces la referencia del objeto es insegura. Por supuesto, esto se puede comprobar o ampliar más allá de los métodos GET para ver datos, pero también para manipularlos.

1.2.1 Observing Differences & Behaviors

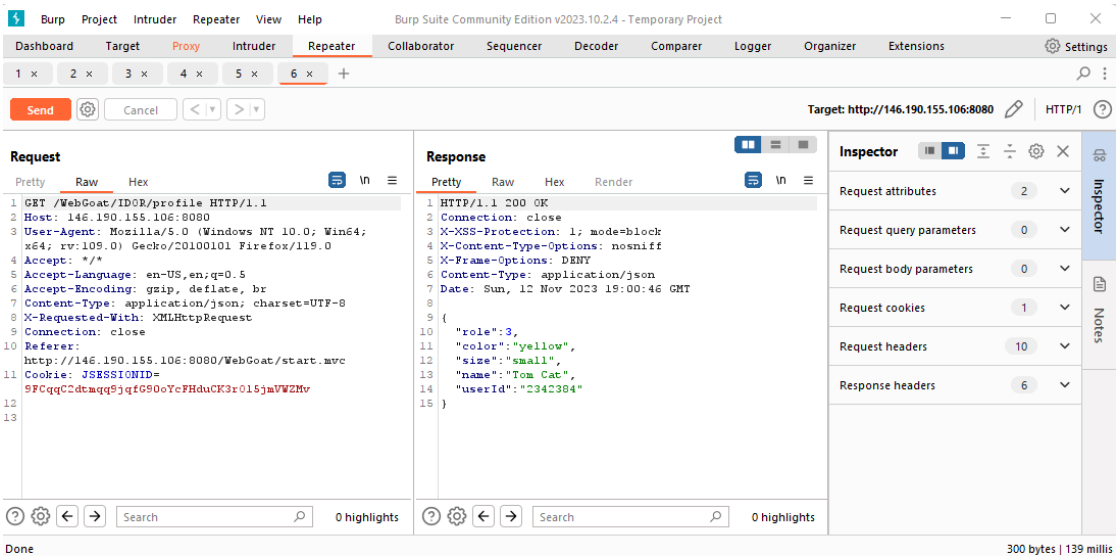
Haz login con el usuario indicado.



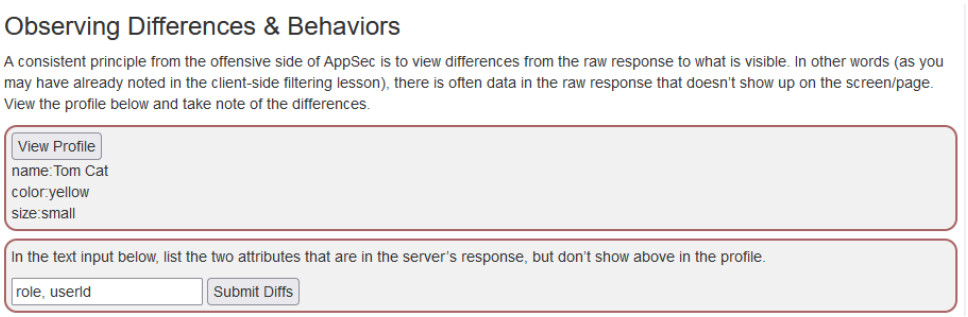
A partir de la lección 3, captura el trafico con el proxy.



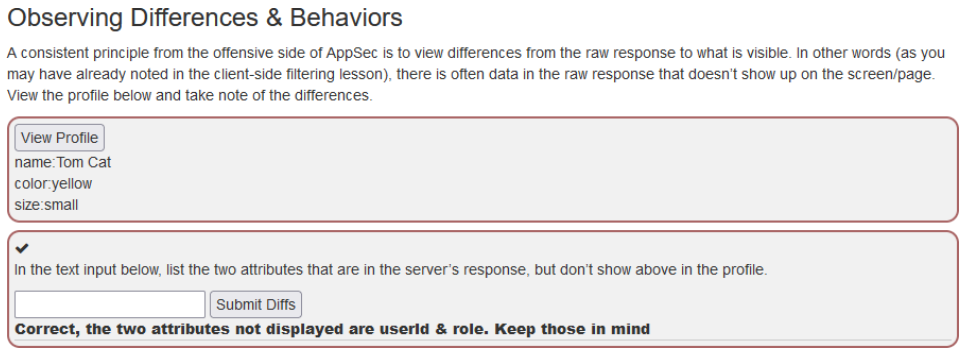
Agrega la captura al "**Repeater**". En Repeater haz clic en "**Send**".



Los campos ocultos que no se muestran en la pagina son: role y userId.



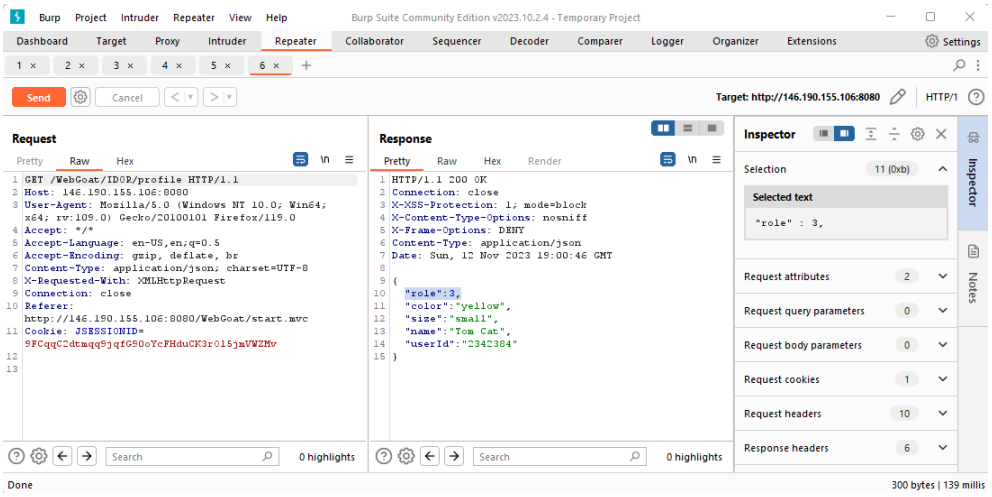
Envia tu respuesta. Haz clic en "**Submit Diffs**".



1.2.2 Guessing & Predicting Patterns

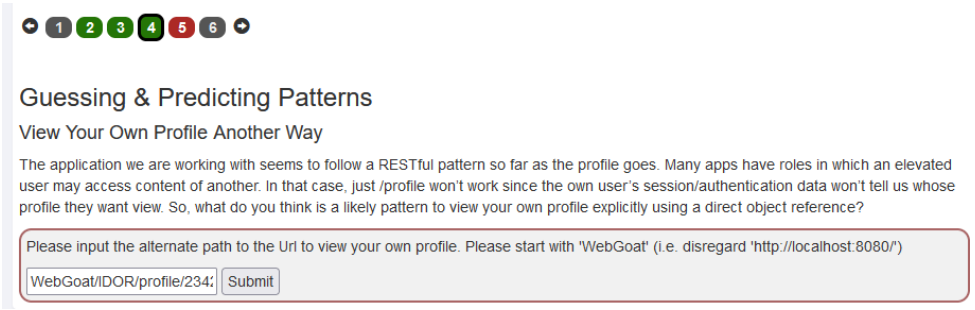
La aplicación con la que estamos trabajando parece seguir un patrón RESTful en lo que respecta al perfil. Muchas aplicaciones tienen funciones en las que un usuario elevado puede acceder al contenido de otro. En ese caso, simplemente /profile no funcionará ya que los datos de sesión/autenticación del propio usuario no nos dirán qué perfil quieren ver. Entonces, ¿cuál crees que es un patrón probable para ver tu propio perfil explícitamente usando una referencia de objeto directa?.

Analizando el "Repeater" del ejercicio anterior.

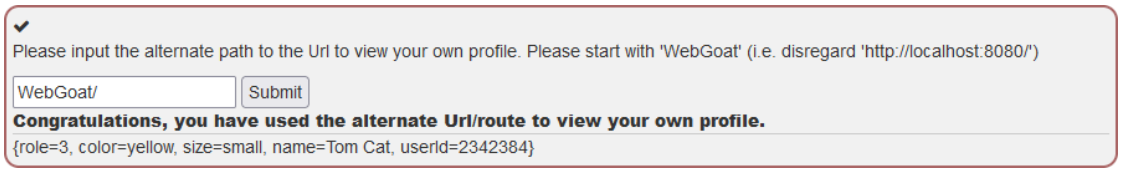


Observamos que el patron es:

/WebGoat/IDOR/profile/2342384



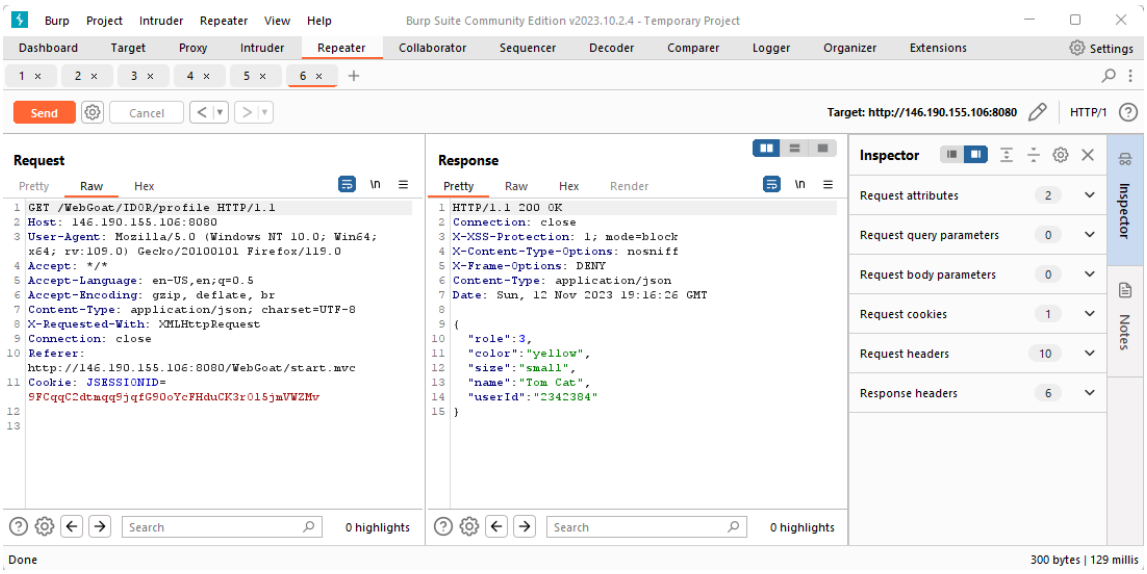
Haz Clic en “Submit”.



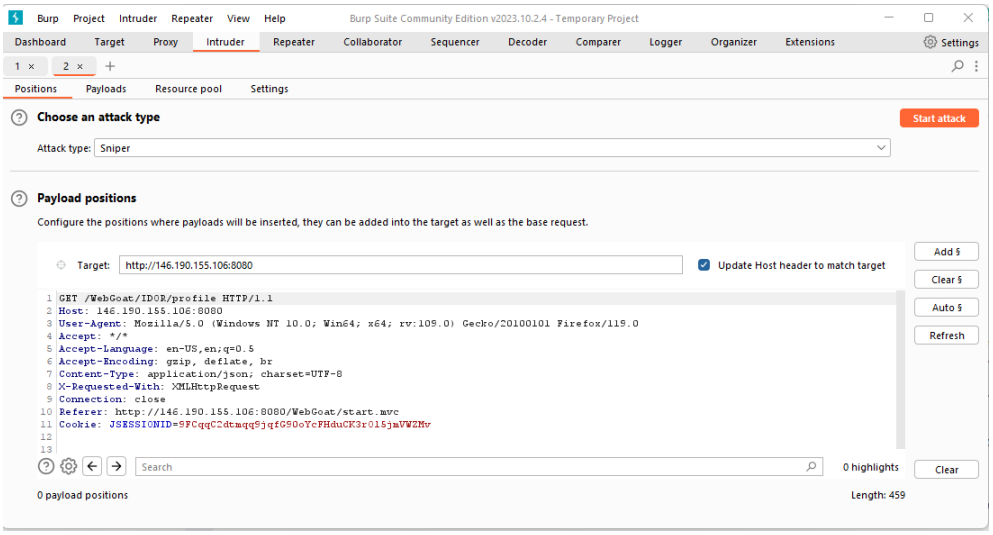
1.2.3 Playing with the Patterns

Vea el perfil de otra persona utilizando la ruta alternativa que ya utilizó para ver su propio perfil. Utilice el botón 'Ver perfil' e intercepte/modifique la solicitud para ver otro perfil. Alternativamente, es posible que también pueda utilizar una solicitud GET manual con su navegador.

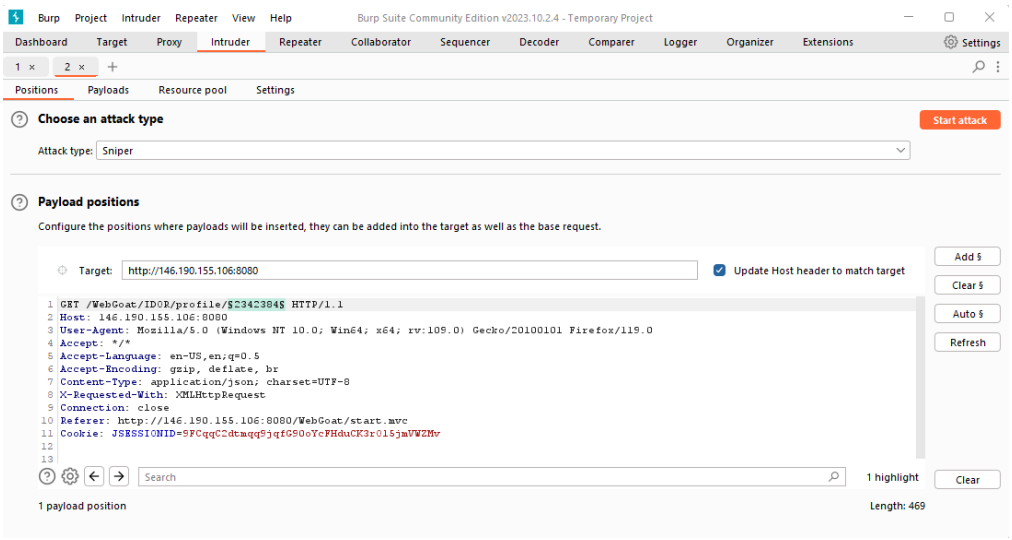
Nuestro perfil lo podemos ver:



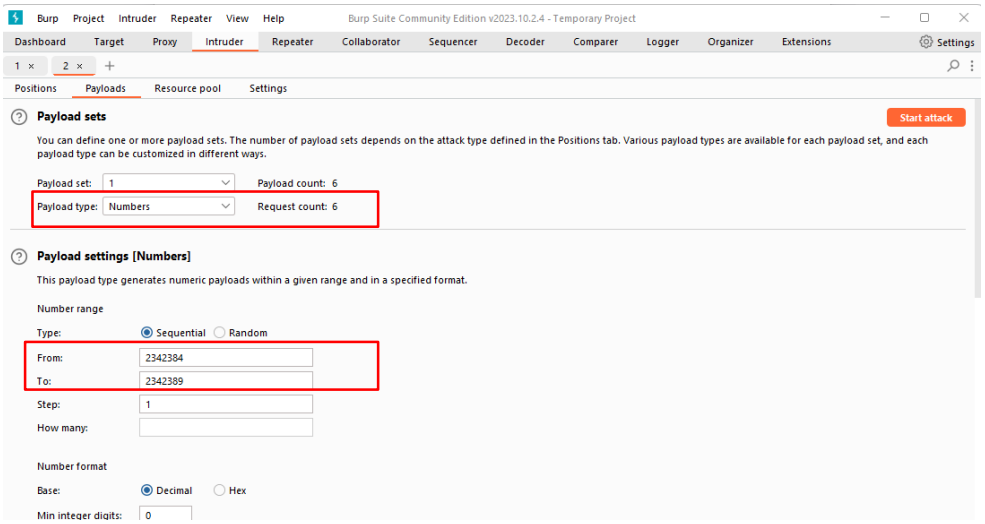
¿Pero podemos ver el perfil de otro usuario?, no deberías poder verlo pero un ataque de intrusión lo podría conseguir. Agrega la petición de “**Repeater**” a “**Intruder**”.



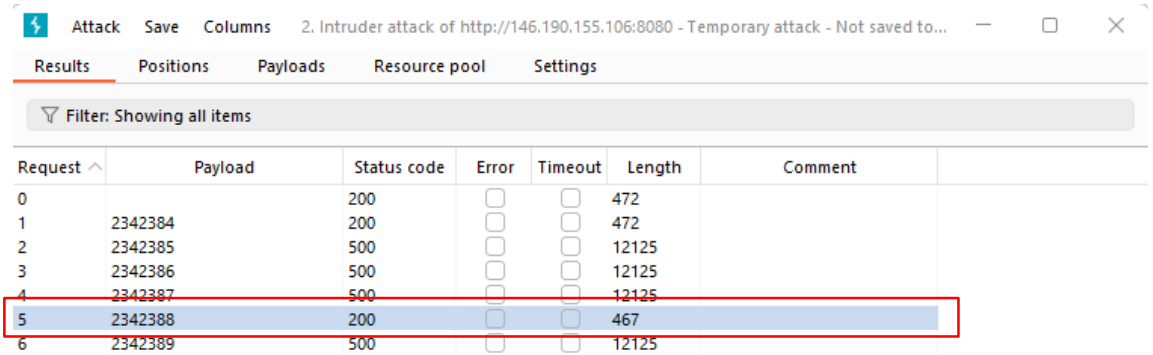
Agrega tu **userId** en “**Intruder**”.

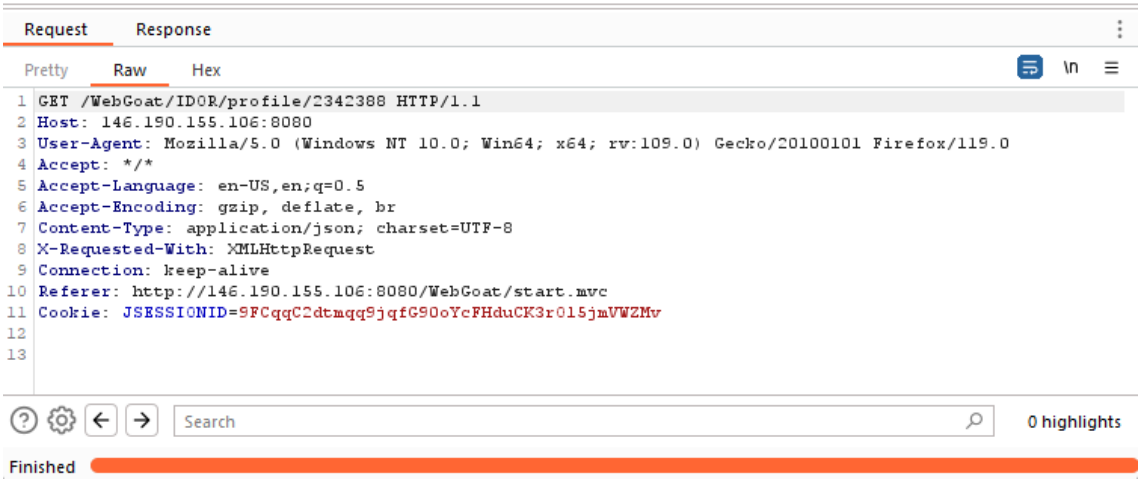


En **Intruder>>Payloads**. En **Payload type** asigna **Numbers**

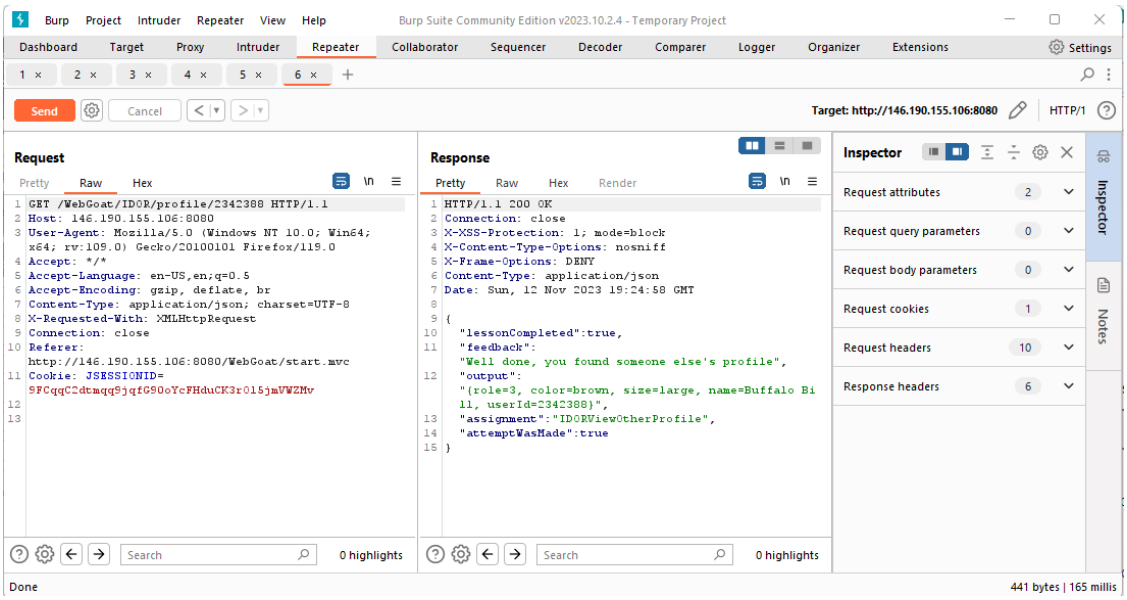


Hz clic en “**Star Atack**” y encuentra un nuevo usuario.



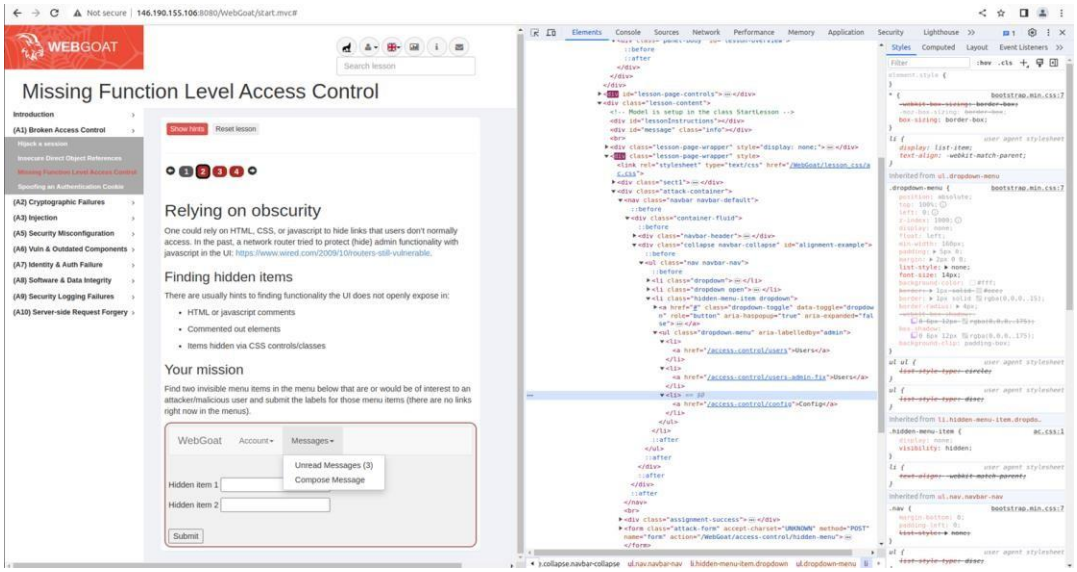


Eureka!, encontramos un usuario con código 234288. Usa “Repeater” para obtener los datos del perfil de este usuario.

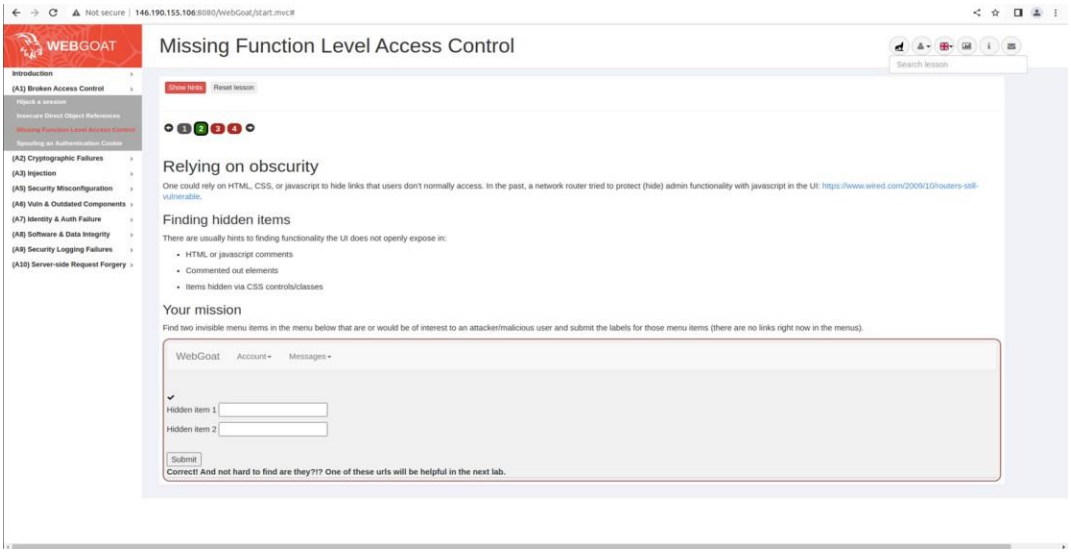


1.3 Missing Function Level Access Control

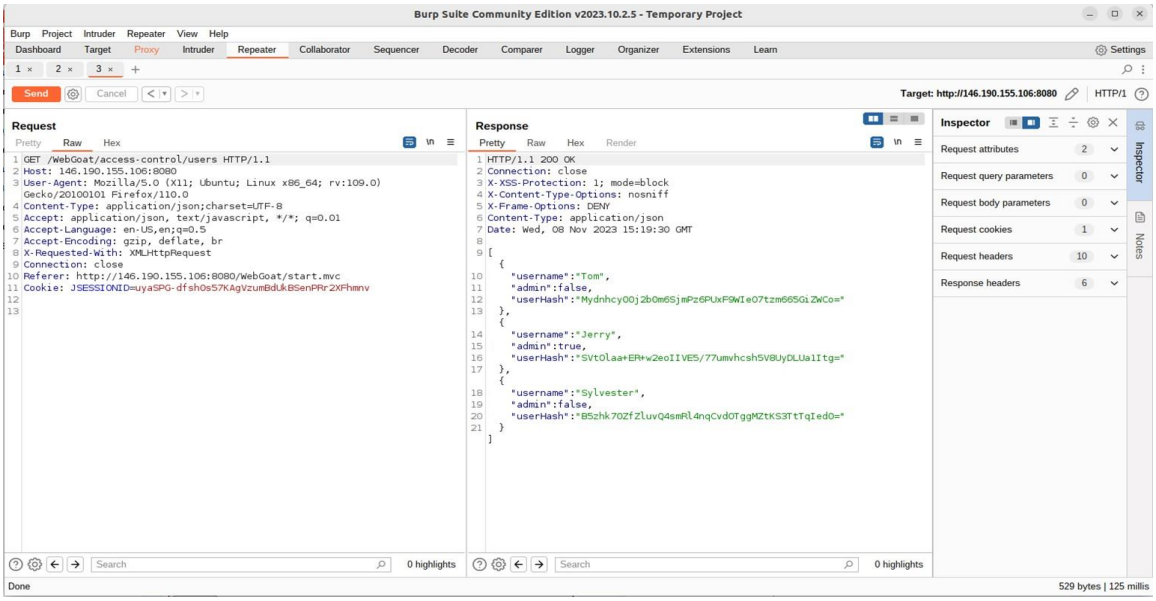
Los parámetros ocultos son **Users** y **Config**



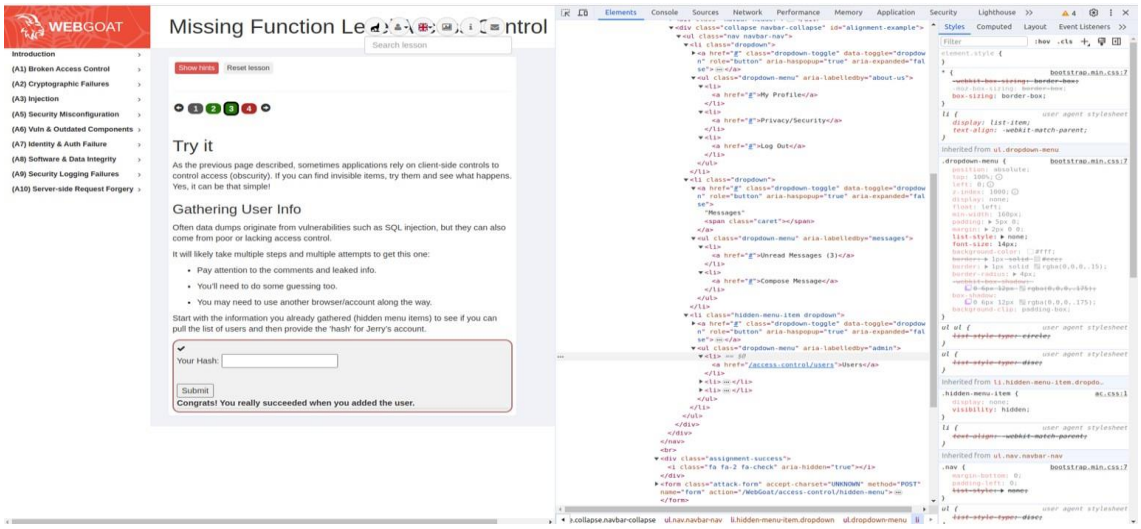
Verifica introduciendo los parámetros en el formulario.



Encontrar el hash del usuario.



Copia y pega el hash del usuario **Jerry**.



1.4 Spoofing an Authentication Cookie

Las cookies de autenticación se utilizan para servicios que requieren autenticación, cuando el usuario inicia sesión con un nombre de usuario y contraseña personales, el servidor valida las credenciales proporcionadas y, si son válidas, crea una sesión.

Cada sesión suele tener un ID único que identifica la sesión del usuario; cuando el servidor devuelve la respuesta al usuario, incluye un encabezado Set-Cookie que contiene, entre otras cosas, el nombre y el valor de la cookie.

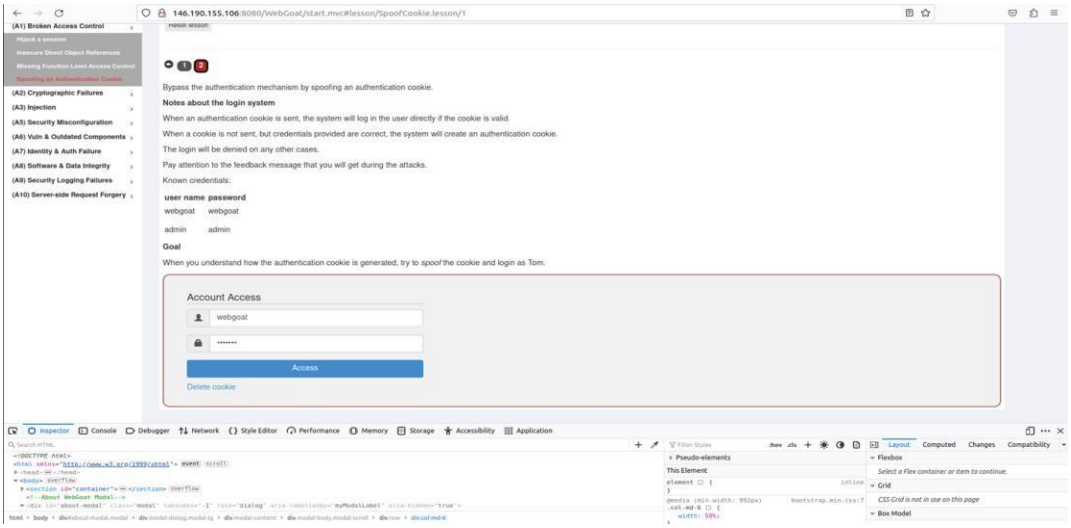
La cookie de autenticación normalmente se almacena en el lado del cliente y del servidor.

Por un lado, tener la cookie almacenada del lado del cliente implica que puede ser robada aprovechando determinadas vulnerabilidades o interceptada mediante ataques man in the middle o XSS. Por otro lado, los valores de las cookies se pueden adivinar si se puede obtener el algoritmo para generarlas.

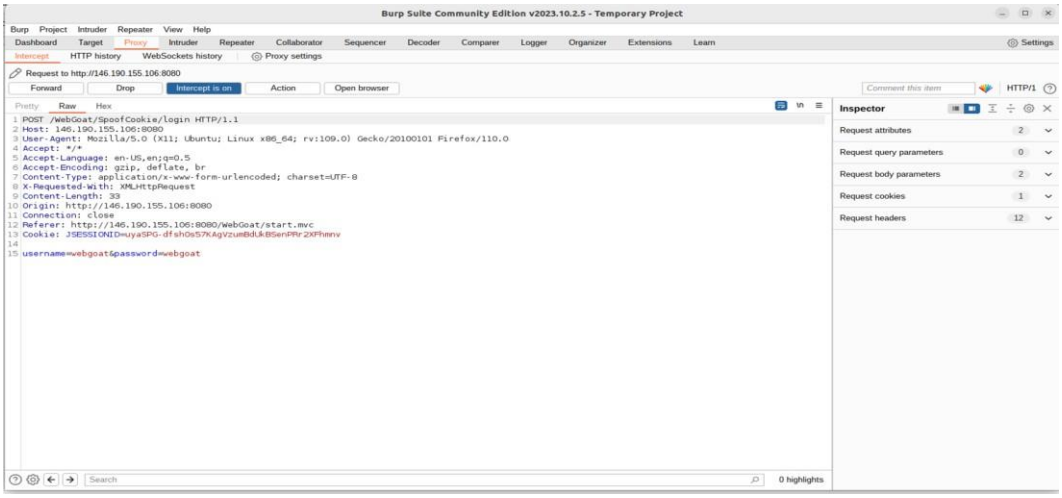
Muchas aplicaciones iniciarán automáticamente la sesión de un usuario si se proporciona la cookie de autenticación correcta.

El usuario debería poder adivinar el algoritmo de generación de cookies y evitar el mecanismo de autenticación iniciando sesión como un usuario diferente.

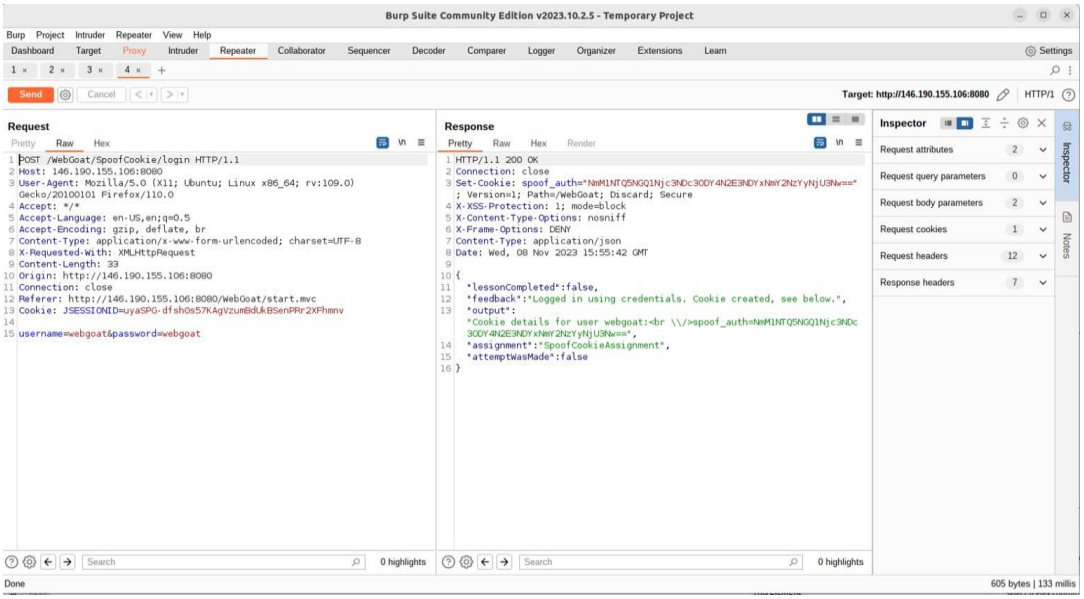
Ingresa con el usuario webgoat clave webgoat, captura la petición con Burp suite.



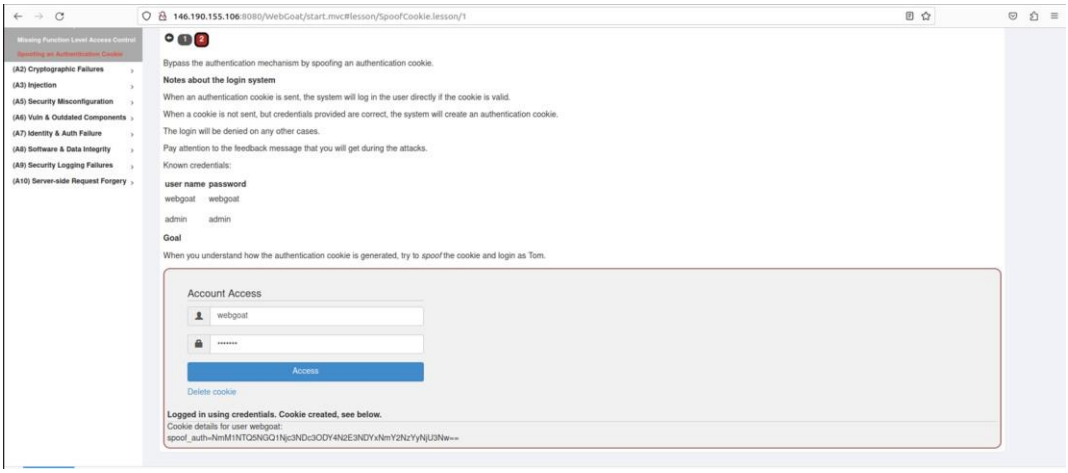
Verifica la petición capturada en Burp suite.



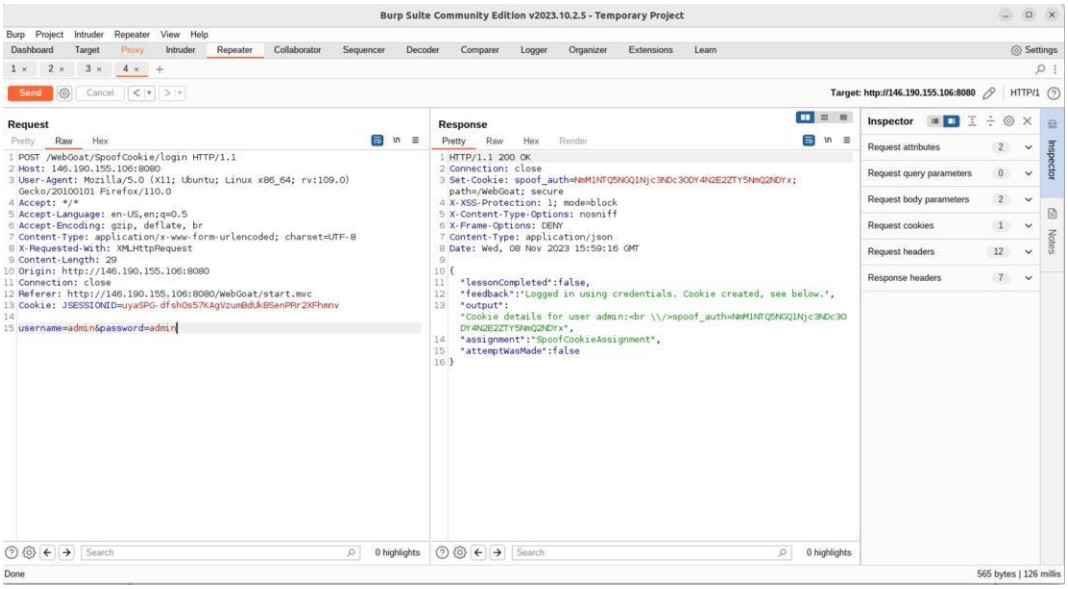
Agrega al “**Repeater**” la petición capturada.



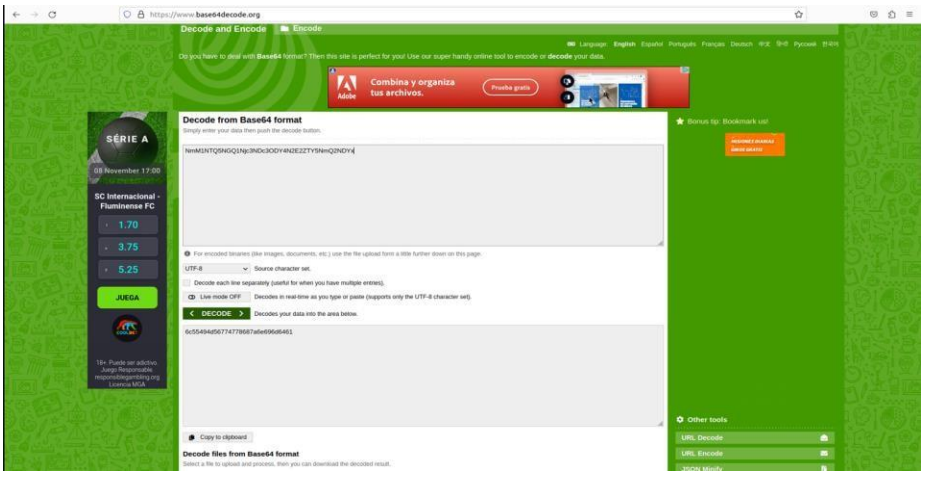
Haz Clic en “**Forward**” en el Proxy y verifica que se ha ingresado correctamente.



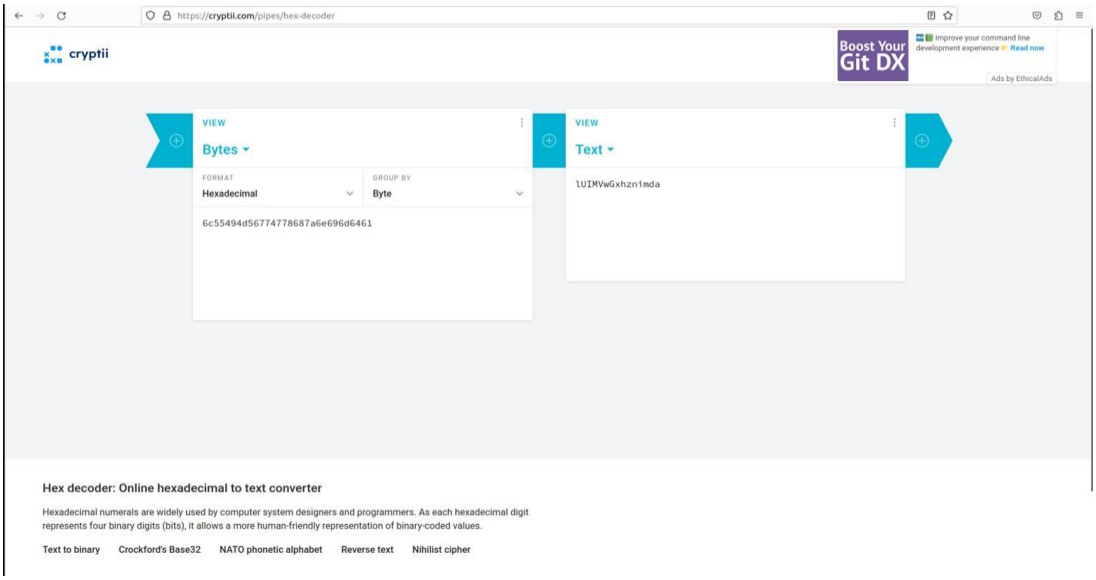
En el “**Repeater**” cambia las credenciales para ingresar con el usuario **admin**.



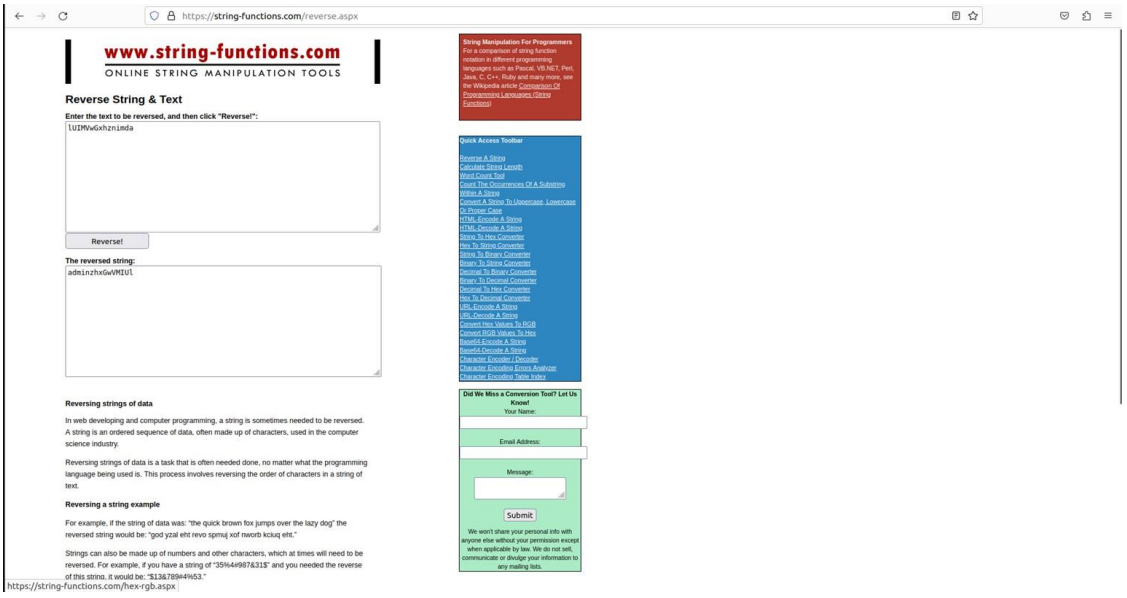
Decodifica la cookie.



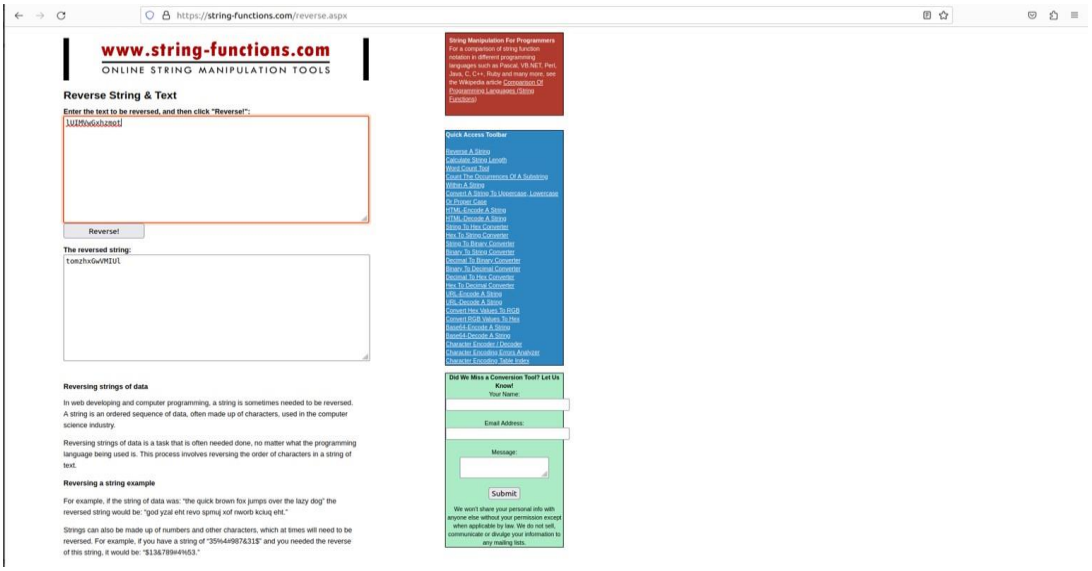
Convierte a cadena ASCII los códigos hexadecimales.



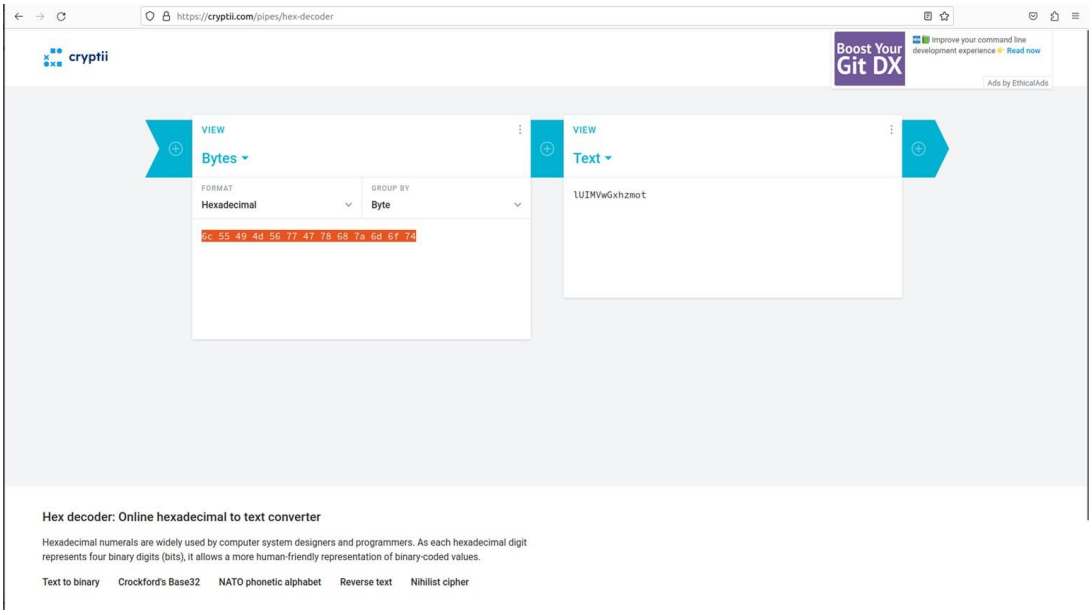
Convierte a reverse la cadena generada en el paso previo.



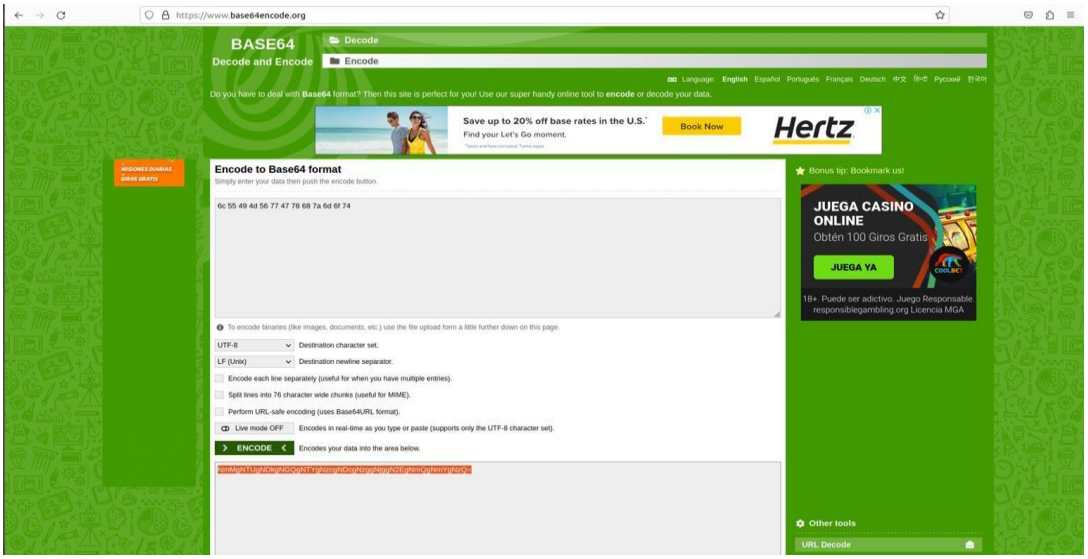
Finalmente podemos ganar acceso con otra cuenta si cambiamos el Cookie usando la cuenta tom. Obtén la cadena en reversa con la cuenta tom.



Decodifica para obtener los caracteres hexadecimales.



Obtén la Cookie para tom.



Finalmente prueba con Burp suite el login exitoso, con la cuenta tom.

